NAME:- PARVINDER KAUR

UID:- 23BCS11591

CLASS:- 23BCS-KRG-1-B

## EXPERIMENT-7

Ques:- Cheapest flights within k stops.

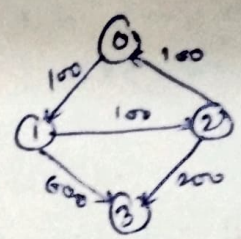
There are n cities connected by some no. of flights.

Given: flights[i] = [from_i, to_i, price_i], src, dest, k

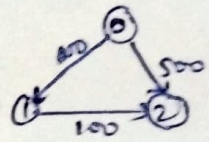
Return cheapest price from src to dest with atmost k stops, else return -1.

I/P 1:- n=4, flights = $\begin{bmatrix} 0,1,1000 \\ 1,2,100 \\ 2,0,100 \\ 1,3,600 \\ 2,3,200 \end{bmatrix}$ src=0, dest=3, k=1

O/P:- 700




I/P 2:- n=3, flights = $\begin{bmatrix} 0,1,1000 \\ 1,2,100 \\ 0,2,500 \end{bmatrix}$, src=0, dest=2, k=1




IDEA:- We need min. cost but with atmost k stops. Use BFS level order where each level=1 stop. Also maintain distance array to store min. cost. This is BFS + Cost Relaxation.

ALGO:-

1) Make adjacency list
2) Queue → {stops, node, cost}
3) Distance array → min cost to each node.
4) Push {0, src, 0}
5) While queue is not empty:
   i) if stops > k → continue
   ii) for neighbours:
      - if newcost < dist → update & push
6) Return dist[dest] or -1.

PSEUDOCODE:

```
func. cheapestFlights (n, flights, src, dest, k):
    adj = adjacency list
    for each flight (u, v, price):
        adj[u].push(v, price)

    dist = array of size n & fill dist with Infinity.
    dist[src] = 0

    queue q
    q.push(0, src, 0)
    while (!q.empty()):
        (stops, node, cost) = q.pop()
        if stops > k:
            continue
        for each (adjNode, price) in adj[node]:
            if cost + price < dist[adjNode]:
                dist[adjNode] = cost + price
                q.push(stops+1, adjNode, dist[adjNode])

    if dist[dest] == Infinity:
        return -1
    else return dist[dest].
```

TIME COMPLEXITY :- $O(K * E)$

SPACE COMPLEXITY :- $O(N+E)$